

## Source Code :-

```
import os

import copy

import time

import matplotlib.pyplot as plt


import torch

import torch.nn as nn

import torch.optim as optim

from torch.optim import lr_scheduler

from torch.utils.data import DataLoader

from torchvision import datasets, models, transforms


from sklearn.metrics import confusion_matrix, classification_report

import seaborn as sns


# =====

# CONFIG

# =====

DATA_DIR = "chest_xray"

BATCH_SIZE = 8

NUM_EPOCHS = 10

NUM_CLASSES = 2

DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")


# =====

# TRANSFORMS

# =====
```

```

data_transforms = {
    'train': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ]),
}

# =====
# LOAD DATA
# =====

image_datasets = {
    x: datasets.ImageFolder(os.path.join(DATA_DIR, x), data_transforms[x])
    for x in ['train', 'val']
}

dataloaders = {
    x: DataLoader(image_datasets[x], batch_size=BATCH_SIZE, shuffle=True)
    for x in ['train', 'val']
}

```

```

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes

# =====

# MODEL

# =====

model = models.resnet18(pretrained=True)

for param in model.parameters():
    param.requires_grad = True

num_fts = model.fc.in_features
model.fc = nn.Linear(num_fts, NUM_CLASSES)

model = model.to(DEVICE)

# =====

# LOSS + OPTIMIZER

# =====

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0003)
scheduler = lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

# =====

# TRAIN FUNCTION

# =====

def train_model(model, num_epochs):
    best_acc = 0.0
    best_model_wts = copy.deepcopy(model.state_dict())

```

```
train_acc_history = []
```

```
val_acc_history = []
```

```
for epoch in range(num_epochs):
```

```
    print(f"Epoch {epoch+1}/{num_epochs}")
```

```
    print("-" * 10)
```

```
    for phase in ['train', 'val']:
```

```
        if phase == 'train':
```

```
            model.train()
```

```
        else:
```

```
            model.eval()
```

```
    running_corrects = 0
```

```
    for inputs, labels in dataloaders[phase]:
```

```
        inputs = inputs.to(DEVICE)
```

```
        labels = labels.to(DEVICE)
```

```
    optimizer.zero_grad()
```

```
    with torch.set_grad_enabled(phase == 'train'):
```

```
        outputs = model(inputs)
```

```
        _, preds = torch.max(outputs, 1)
```

```
        loss = criterion(outputs, labels)
```

```
    if phase == 'train':
```

```
        loss.backward()
```

```

        optimizer.step()

    running_corrects += torch.sum(preds == labels.data)

    if phase == 'train':
        scheduler.step()

    epoch_acc = running_corrects.double() / dataset_sizes[phase]
    print(f"{phase} Acc: {epoch_acc:.4f}")

    if phase == 'val':
        val_acc_history.append(epoch_acc.cpu().numpy())

        if epoch_acc > best_acc:
            best_acc = epoch_acc

            best_model_wts = copy.deepcopy(model.state_dict())
        else:
            train_acc_history.append(epoch_acc.cpu().numpy())

    print()

    model.load_state_dict(best_model_wts)
    return model, train_acc_history, val_acc_history

# =====

# TRAIN

# =====

model, train_acc, val_acc = train_model(model, NUM_EPOCHS)

# =====

```

```

# PLOT

# =====

plt.plot(train_acc, label="Train")
plt.plot(val_acc, label="Val")
plt.legend()
plt.title("Accuracy")
plt.savefig("accuracy.png")
plt.show()

# =====

# CONFUSION MATRIX

# =====

all_preds = []
all_labels = []

model.eval()
with torch.no_grad():
    for inputs, labels in dataloaders['val']:
        inputs = inputs.to(DEVICE)
        labels = labels.to(DEVICE)

        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)

```

```
sns.heatmap(cm, annot=True)
```

```
plt.title("Confusion Matrix")
```

```
plt.savefig("cm.png")
```

```
plt.show()
```

```
print(classification_report(all_labels, all_preds))
```

```
# =====
```

```
# SAVE MODEL
```

```
# =====
```

```
torch.save(model.state_dict(), "model.pth")
```

```
print("Model saved!")
```